BreaQ 2026 — Quantum Hacking Track

Challenge 1: *Trust No Input* — Solution Write-up

Team: `DEMO-TEAM`

24 October 2026

Abstract

A quantum policy-fingerprint service rejects valid assets after a migration, and Operations blames the quantum circuit. We show experimentally that the circuit and the ideal backend are correct: every case produces the registry fingerprint deterministically (512/512 shots, statevector probability 1). The fault lies in the application layer. A legacy `compatibility_map`, taken from an untrusted imported manifest, permutes the decoded fingerprint *before* the authorization comparison. We give a minimal reproducer, a canonical decoder that ignores presentation metadata (112/112 tests passed, including fingerprints absent from the instance), and an exploit that turns the same flaw into a *false accept* whenever source and target have equal Hamming weight.

Security claim (one sentence). *An attacker who can edit only the `compatibility_map` of an imported manifest can flip the service's `MATCH`/`MISMATCH` decision without touching the quantum circuit, because the decision compares the registry value against a metadata-permuted string instead of the canonical $q_0 \dots q_3$ measurement.*

Root-cause layer. Application (report adapter), not circuit construction and not quantum noise.

1 Background

1.1 System under investigation

```
Asset registry -- expected fingerprint
                          |
Manifest --> circuit builder --> ideal backend --> raw counts
   |                                                  |
   +-------- compatibility metadata --> report adapter --> MATCH / MISMATCH
```

The contract states:

- four logical slots, ordered `alpha, beta, gamma, delta`;
- slot i is encoded on Qiskit qubit q_i;
- the registry fingerprint is written in canonical slot order;
- legacy compatibility metadata may affect *presentation*, but must not silently change a *security decision*.

1.2 The Bernstein–Vazirani probe

For a secret string $s \in \{0,1\}^n$, the oracle computes $f(x) = s \cdot x \bmod 2$. With the ancilla prepared in $|-\rangle = HX|0\rangle$, each $\mathtt{CX}(q_i, \text{anc})$ for $s_i = 1$ applies phase kickback, and the final Hadamard layer gives

$$H^{\otimes n} \frac{1}{\sqrt{2^n}} \sum_x (-1)^{s \cdot x} |x\rangle \;=\; |s\rangle.$$

On an ideal simulator, then, a single run returns s with probability 1. Any spread in the counts would point to noise; a sharp, wrong value would point to construction or interpretation.

1.3 Qiskit bit order

Qiskit numbers qubits from q_0 upward, but a count key is *displayed* with the highest classical bit on the left. A displayed key $k = k_{n-1} \dots k_1 k_0$ therefore maps to canonical order as

$$\text{canonical}(k) = k_0\, k_1 \dots k_{n-1} = \text{reverse}(k).$$

For the two-qubit warm-up, X on q_0 yields the displayed key `01`: the rightmost character is q_0.

Warm-up answer. The displayed key `0110` reversed is `0110`. This key is a palindrome, so the two orders coincide, and that makes it a poor test of whether an implementation reverses at all. A key such as `0101` → `1010` does reveal the difference.

2 Threat model

Protected asset	Integrity of the `MATCH`/`MISMATCH` authorization decision.
Required property	The decision depends only on the measured (canonical) fingerprint and the trusted registry value.
Adversary capability	Can supply or modify an imported manifest, specifically its `compatibility_map`; cannot alter the circuit, the backend or the registry.
Trusted inputs	Registry fingerprint; circuit builder; ideal backend and its raw counts.
Untrusted inputs	Manifest metadata: `schema`, `compatibility_map`, `provenance`.
Observable symptom	Valid assets rejected after migration (`MIG-DELTA`, `INCIDENT-7`).
Alternative explanations	(i) quantum noise; (ii) wrong oracle or circuit; (iii) bit-order confusion when reading counts; (iv) unsafe application-layer reinterpretation.

3 Phase A — Triage

All four cases were executed with 512 shots (seed 2026). "Canonical" is the reversed dominant key.

Case	Registry	Raw key	Canonical	Map	Reported	Decision
`CAL-ALPHA`	`1001`	`1001` (512)	`1001`	$[0,1,2,3]$	`1001`	`MATCH`
`CAL-BETA`	`0110`	`0110` (512)	`0110`	$[0,1,2,3]$	`0110`	`MATCH`
`MIG-DELTA`	`1010`	`0101` (512)	`1010`	$[1,0,2,3]$	`0110`	`MISMATCH`
`INCIDENT-7`	`1100`	`0011` (512)	`1100`	$[3,1,2,0]$	`0101`	`MISMATCH`

Observations:

1. Every case is deterministic (one key, 512/512 shots), so there is no sign of noise.
2. In every case the canonical value equals the registry value, so the circuit encodes the right fingerprint.
3. The two failing cases are exactly the two with a non-identity compatibility map. Both come from legacy-schema manifests (`migration-adapter`, `external-import`).

4 Phase B — Localizing the fault

Each boundary was checked independently.

Boundary 1: prepared circuit. We removed the final measurements and computed the exact `Statevector` marginal on $q_0 \dots q_3$:

Case	Top statevector key	Reversed	P
`CAL-ALPHA`	`1001`	`1001`	1.0
`CAL-BETA`	`0110`	`0110`	1.0
`MIG-DELTA`	`0101`	`1010`	1.0
`INCIDENT-7`	`0011`	`1100`	1.0

The circuit for `MIG-DELTA` contains `CX` gates from q_0 and q_2 to the ancilla q_4, which encodes $s =$ `1010` exactly as the contract requires.

Boundary 2: raw backend output. The raw counts match the statevector prediction exactly. Reversing them recovers the registry value in all four cases.

Boundary 3: application report. The report is the first point where values diverge. The adapter computes

$$\text{reported}[j] = \text{canonical}\big[\,\text{map}[j]\,\big],$$

and `execute` compares *this* string with the registry. For `MIG-DELTA`: canonical `1010`, map $[1, 0, 2, 3]$, giving $c_1c_0c_2c_3 =$ `0110` $\neq$ `1010`.

4.1 Causal claim and controlled experiment

Claim. *Changing only the* ***`compatibility_map`****, while holding the encoded fingerprint, circuit, backend and seed constant, changes the reported fingerprint and therefore the decision, while the canonical quantum result stays the same.*

Experiment. We fixed $s =$ `1010` and ran `service.simulate_adapter` under all 24 permutations.

Distinct canonical results	{`1010`} (one value, all 24 runs)
Distinct reported results	`1010`, `1001`, `1100`, `0110`, `0101`, `0011` (4 maps each)
Maps that alter the report	20 of 24

The quantum output does not change, and the report covers every weight-2 string. The independent variable (the map) fully explains the dependent variable (the decision). This also rules out noise: the backend returns the same value regardless of the map, and the map never reaches the backend.

5 Phase C — Minimal reproducer

One source fingerprint, two metadata configurations, identical quantum results, different application reports:

Map	Raw counts	Canonical	Reported	vs. registry `1010`
$[0, 1, 2, 3]$	{`0101`: 512}	`1010`	`1010`	MATCH
$[1, 0, 2, 3]$	{`0101`: 512}	`1010`	`0110`	MISMATCH

```
def minimal_reproducer(service):
    fp = "1010"
    configs = {"identity": (0, 1, 2, 3), "legacy_swap": (1, 0, 2, 3)}
    runs = {name: service.simulate_adapter(fp, m) for name, m in configs.items()}
    return {
```

```
        "source_fingerprint": fp,
        "runs": runs,
        "same_canonical": len({r["canonical_fingerprint"] for r in runs.values()}) ==
    1,
        "different_report": len({r["reported_fingerprint"] for r in runs.values()}) >
    1,
    }
```

6 Phase D — Defensive fix

6.1 Design

The authorization path must use only trusted data, in canonical slot order. The legacy permutation is a *display* concern. It may be applied to a human-readable report *after* the decision, but never fed into it.

```
def secure_decode(counts: dict[str, int]) -> str:
    """Return the fingerprint in canonical q0..q(n-1) order.
    Reads nothing but the raw counts: no case ID, map or registry value."""
    if not counts:
        raise ValueError("empty counts")
    key = max(counts, key=counts.get).replace(" ", "")
    if set(key) - {"0", "1"}:
        raise ValueError("malformed count key")
    return key[::-1]          # displayed key is q(n-1)..q0


def secure_decision(counts: dict[str, int], registry_fingerprint: str) -> str:
    measured = secure_decode(counts)
    if len(registry_fingerprint) != len(measured) or \
       set(registry_fingerprint) - {"0", "1"}:
        raise ValueError("malformed registry fingerprint")
    return "MATCH" if measured == registry_fingerprint else "MISMATCH"
```

Applied to the four instance cases, `secure_decision` returns `MATCH` for all of them, including the two incidents.

6.2 Tests

The tests deliberately go beyond the demonstration instance:

- **Positive:** all 16 four-bit fingerprints (12 of which are absent from `team_instance.json`). Each is run under the identity map plus 6 random permutations, and must decode to itself and give `MATCH`.
- **Negative:** for each run, the bitwise complement of the fingerprint is used as the registry value and must give `MISMATCH`.
- **Metadata invariance:** because the map is varied while the expected output is fixed, any dependence of the decoder on the map would fail a test.

```
def run_defense_tests(seed=7):
    rng = random.Random(seed)
    perms = list(permutations(range(4)))
    passed = failed = 0
    for fp in map("".join, product("01", repeat=4)):
        for m in rng.sample(perms, 6) + [(0, 1, 2, 3)]:
            counts = service.simulate_adapter(fp, m, shots=128)["counts"]
            wrong = format(int(fp, 2) ^ 0b1111, "04b")
            ok = (secure_decode(counts) == fp
```

```
                and secure_decision(counts, fp) == "MATCH"
                and secure_decision(counts, wrong) == "MISMATCH")
            passed += ok
            failed += not ok
    return {"implemented": True, "passed": passed, "failed": failed}
```

Result. `{"implemented": True, "passed": 112, "failed": 0}`.

7 Phase E — Exploit

7.1 Reachability

A permutation only *reorders* bits, so it preserves the Hamming weight $w(\cdot)$. Hence

$$\exists\, \pi \in S_n : \ \pi(\text{source}) = \text{target} \iff w(\text{source}) = w(\text{target}).$$

($\Rightarrow$) is immediate. For ($\Leftarrow$), pair the i-th `1` of the target with the i-th `1` of the source, and likewise for the `0`s. From a source of weight w, an attacker can therefore reach exactly $\binom{n}{w}$ reports, and nothing outside that set.

```
def craft_compatibility_map(source: str, target: str):
    if len(source) != len(target) or sorted(source) != sorted(target):
        return None                          # Hamming weights differ: unreachable
    pools = {b: [i for i, x in enumerate(source) if x == b] for b in "01"}
    return tuple(pools[b].pop(0) for b in target)
```

7.2 Verification via `simulate_adapter`

Source	Target	w	Crafted map	Verified report	Outcome
1010	0110	2/2	$(1,0,2,3)$	0110	reachable
1010	1100	2/2	$(0,2,1,3)$	1100	reachable
0110	1001	2/2	$(1,0,3,2)$	1001	reachable
1001	0110	2/2	$(1,0,3,2)$	0110	reachable
1010	1110	2/3	None	—	impossible
1100	0000	2/0	None	—	impossible

7.3 Impact: from false reject to false accept

The customer reported *false rejects*, but the same mechanism yields *false accepts*. Consider an asset whose circuit encodes `1010` and an imported manifest that carries map $(1,0,2,3)$. The service reports `0110`, which is exactly `CAL-BETA`'s registry fingerprint, so the wrong asset is accepted as `CAL-BETA`. In general, any asset can impersonate any registry entry of equal Hamming weight. The all-zeros and all-ones fingerprints are the only ones immune, since they are fixed by every permutation.

8 Research extension — exhaustive analysis

We tested all 16 fingerprints against all 24 permutations through the service. A fingerprint of weight w is left unchanged by exactly the permutations that shuffle its `1`s among themselves and its `0`s among themselves, i.e. by $w!\,(n-w)!$ of them. Therefore

$$\#\{\pi : \pi(s) \neq s\} \ = \ n! - w!\,(n-w)!.$$

Weight w	Fingerprints	Altering perms (each)	Predicted $24 - w!(4-w)!$
0	1	0	$24 - 24 = 0$
1	4	18	$24 - 6 = 18$
2	6	20	$24 - 4 = 20$
3	4	18	$24 - 6 = 18$
4	1	0	$24 - 24 = 0$
Total altering pairs		264 of 384 (68.75%)	

The measured values match the formula exactly.

Generalization to n bits. `secure_decode` is already n-agnostic: reversing the key is $O(n)$. `craft_compatibility_map` is also $O(n)$. For a balanced fingerprint ($w = n/2$), the fraction of permutations that alter it, $1 - \left(\frac{n}{2}!\right)^2/n! = 1 - 1/\binom{n}{n/2}$, tends to 1. Almost every non-trivial map therefore changes the decision.

Manifest-validation policy. Separate the two kinds of semantics:

- *Security semantics* (slot order, fingerprint encoding) are fixed by the contract and are never read from a manifest.
- *Presentation semantics* (`compatibility_map`) are applied only to a display string produced after the decision.

Checking that the map "is a permutation" (which the service already does) is insufficient: every malicious map in this report is a valid permutation. Validation constrains form, not intent.

Authenticated binding. Signing the manifest (e.g. HMAC over the canonical JSON) prevents tampering in transit. It does *not* help if the signer itself emits a wrong legacy map, if a valid signed map is replayed for another asset, or, most importantly, if the decision path still consumes the map. The real fix is to remove the map from the security path. Authentication is defense in depth.

9 Claim sheet

Claim	**Evidence**	**Assumption**	**Limitation**
The fault is *not* quantum noise.	512/512 shots on one key per case; identical canonical result under all 24 maps.	Ideal `BasicSimulator`, as specified.	Real hardware adds noise; the decoder would then need a confidence threshold.
The circuit *does* encode the registry value.	Statevector $P = 1$ on reversed registry value for all four cases.	Contract: slot i on qubit q_i.	Verified for the four instance cases plus all 16 fingerprints via the adapter.
Metadata *can* alter the decision.	Minimal reproducer; 20/24 maps change the report for `1010`.	Attacker controls only the map.	Reachable only within equal Hamming weight.
The mitigation generalizes.	112/112 tests over all 16 fingerprints, random maps, negative complements.	Dominant-key decoding is valid (deterministic output).	Under noise, majority decoding alone may be insufficient.

10 Oral-defense notes

Why is the dominant count key not the canonical fingerprint?

Qiskit displays the highest classical bit on the left, so the key is $q_3q_2q_1q_0$ and must be reversed

to obtain slot order.

Which experiment excludes noise?
Determinism (512/512) together with the 24-map sweep: the canonical result never changes, while the report changes with the map alone.

Can every target be produced by permutation alone?
No. Only targets with the same Hamming weight as the source can be produced, which gives $\binom{n}{w}$ reachable reports.

Why is "is a permutation" validation insufficient?
It checks syntax, not authority. Every attack map is a valid permutation.

Where should legacy presentation conversion occur?
After the decision, on a display copy only, never on the comparison path.

A fresh fingerprint and map from the jury?
`secure_decode` reads only the counts, so it returns the true fingerprint for any input. The exploit function returns a crafted map, or `None` when the Hamming weights differ.

Team member contributions

- **Investigator:** *(name)* — triage, threat model, localization.
- **Exploit developer:** *(name)* — reproducer, crafted maps, exhaustive analysis.
- **Defense analyst:** *(name)* — secure decoder, test suite, validation policy.

References

- IBM Quantum, *Bit ordering in Qiskit*. https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering
- IBM Quantum, `Statevector.probabilities_dict`. https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector
- Qiskit `BasicSimulator` API. https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.providers.basic_provider.BasicSimulator
- E. Bernstein and U. Vazirani, "Quantum complexity theory," *SIAM J. Comput.* 26(5), 1997.